MIT 8103 Advanced Database Systems

Practical Database Portfolio – LMS Documentation

Student	Precious Siyanbola	**Student ID**	301926114
Programme	Master of Information Technology	**Session**	2026/2027 First Semester CA
Case study	VoltGrid Power (simulated electricity distribution company)	**GitHub**	github.com/<your-username>/advanced-database-portfolio

1. Selected organisation

VoltGrid Power is a simulated electricity distribution company serving Lagos, Ogun and Oyo through eight business units, 120 feeders and 1,200 transformers. Each feeder has a tariff band (A–E) that sets both the promised hours of supply and the price per kWh. Prepaid customers buy tokens through the app, banks, POS, USSD and agents; postpaid customers are read daily and billed monthly. **Users:** customers, vending partners, billing officers, revenue-protection and field technicians, the control room, finance and management. **Major data:** business units, feeders, transformers, dated tariff versions, customers, meters, token sales, meter readings, bills, payments, outages and field work orders. **Key operations:** sell exactly one token per payment at the right tariff, read and bill postpaid meters, take payments from several channels, log outages and report hours of supply, and find meters that may be bypassed. A reproducible synthetic data set (40,000 customers, 42,000 meters, 823,792 token sales, 2.64 million readings, 85,032 bills, 39,763 outages) is used in all five activities.

2. Portfolio activities

Portfolio 1 – Design and modelling. The ERD was implemented as 11 PostgreSQL tables in 3NF/BCNF, with the readings table range-partitioned by month. A pricing trigger sells only to active prepaid meters and computes VAT, kWh and the tariff version valid that day; a unique payment reference makes vending idempotent; exclusion constraints stop overlapping tariff versions and overlapping outages; CHECK constraints stop overpaid and duplicate bills. All 13 invalid operations were rejected, a valid sale was priced by the database, and the 0.8 million seeded sales showed 0 violations of seven business rules.

Portfolio 2 – Query processing and optimisation. Five queries were measured with EXPLAIN (ANALYZE, BUFFERS) over three phases (15 plans saved). Indexing the meter column of sales cut the app query from 48 ms to 0.01 ms; a materialised view served the dashboard in 0.1 ms instead of 1.66 s. Rewriting a date_trunc() filter as a range let the planner prune 13 partitions to one (7×). Round 1 did not help Q4, whose plan aggregated every sale; rewriting it as an index anti-join gave 3.6×. A query rewritten to match the outage exclusion constraint used that constraint's GiST index.

Portfolio 3 – Transactions and concurrency. Six transaction cases show a sale and its token SMS committed together (transactional outbox), a reversed payment rolled back, an overpayment aborting the payment row as well, savepoints in a bank settlement file and a tariff version change that only works in the right order. Six scenarios with real concurrent sessions showed a duplicate bank notification producing an error with check-then-insert but the same token with ON CONFLICT; a lost update of ₦100,000 fixed by an atomic UPDATE, with the CHECK stopping a racing overpayment; a tariff change between quote and payment (READ COMMITTED vs REPEATABLE READ); NOWAIT and lock_timeout; DETACH PARTITION stalling meter inserts for 1,963 ms versus 7 ms with CONCURRENTLY; and a month-end report that only adds up under REPEATABLE READ.

Portfolio 4 – NoSQL and advanced data models. Redis 7 rate-limits token requests (30 simultaneous requests: 5 allowed), filters duplicate payment notifications with SET NX, records which of 7,087 meters reported on each of 7 days in 7,560 bytes of bitmaps (BITOP AND matched SQL exactly), serves a live feeder status board and keeps revenue counters. Field work orders, whose content depends on the job type, were stored as 11,592 JSONB documents with CHECK rules; a GIN index cut a query from 3.2 ms to 0.49 ms. The same documents were queried through the MongoDB API (aggregation pipelines, $unwind, a status-guarded update) and matched JSONB.

Portfolio 5 – Distributed and cloud databases. A five-server PostgreSQL 16 lab hash-shards meter readings by meter number (49/51 balance) and runs a hot standby plus a standby that applies changes 30 seconds late. Experiments showed single-shard routing, scatter-gather, aggregate push-down (260 → 139 ms, though a single node took 67 ms) and the cost of hash sharding when a shard fails. A DELETE without WHERE reached the normal standby within 200 ms, but all 76,273 payments were recovered from the delayed standby in 9.7 s. Sync-commit cost, a CAP trade-off and a 101 ms failover with no lost rows were measured, and a managed-cloud design is proposed.

3. Selected evidence

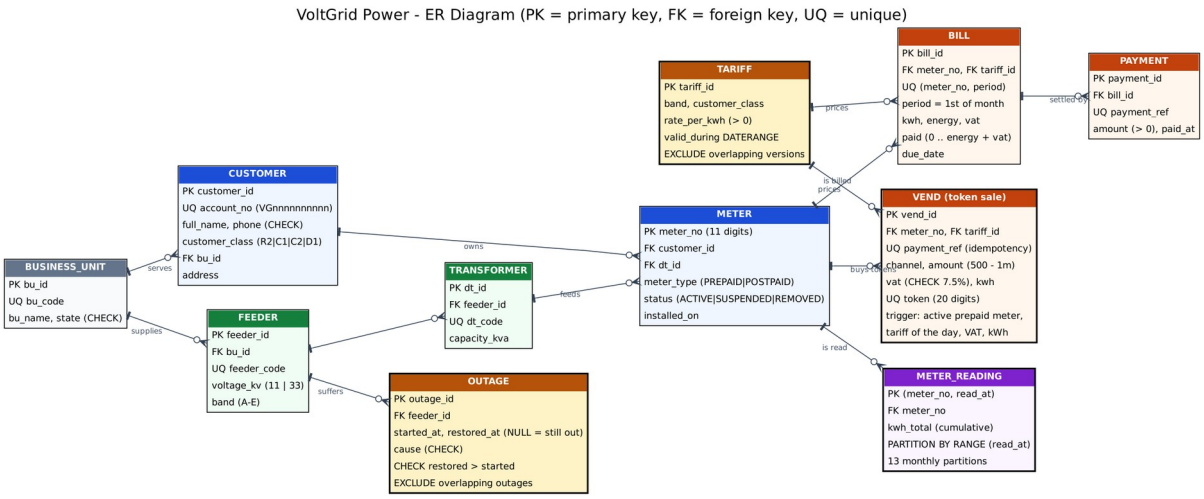


Figure 1. VoltGrid ER diagram (Portfolio 1; source erd.dot in the repository)

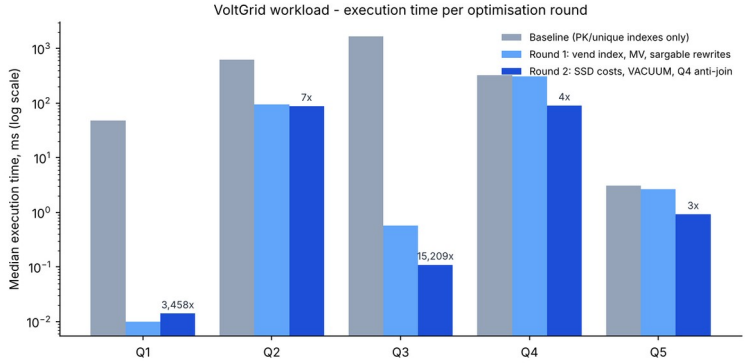


Figure 2. Median execution time per optimisation round, log scale (Portfolio 2)

```
python3 concurrency_demo.py  - outcomes of scenarios S1-S6

S1  DUPLICATE PAYMENT NOTIFICATION - the bank's webhook delivers payment DEMO-WH-xxxx twice, 1 ms apart
RESULT: WH-1: ERROR 23505 vend_payment_ref_key -> customer charged, no token shown | WH-2: token 28531241253941388266 | sales recorded = 1 |
RESULT: WH-1: token 82671210902637905703 | WH-2: token 82671210902637905703 | sales recorded = 1 | both callers got the same token = True
S2  BILL PAYMENT - the bakery pays 100,000 by app and 100,000 at the bank at the same moment (bill total 638,550)
RESULT: {'BANK': 'ok', 'APP': 'ok'} | payments recorded = 200,000.00; bill.paid = 100,000.00 of 638,550.00 => WRONG: 100,000.00 naira paid b
RESULT: {'APP': 'ok', 'BANK': 'ok'} | payments recorded = 200,000.00; bill.paid = 200,000.00 of 638,550.00 => consistent
RESULT: {'APP': 'ok', 'BANK': 'ok', 'POS': 'rejected by CHECK bill_no_overpayment'} | payments recorded = 600,000.00; bill.paid = 600,000.00
S3  TARIFF CHANGES DURING A SALE - Folake (band A, R2) is quoted, then pays; the tariff team commits a new rate in between
RESULT (READ COMMITTED): quoted 41.34 kWh at 225.00; token issued for 37.21 kWh at 250.00 => INCONSISTENT: quoted one price, charged another
RESULT (REPEATABLE READ): quoted 41.34 kWh at 225.00; token issued for 41.34 kWh at 225.00 => consistent
S4  LOCK WAITS - the billing job holds the bakery's bill for 3 s; customer care tries to add a 5,000 naira credit
RESULT (default: wait): got the lock after 3.00 s
RESULT (NOWAIT): ERROR 55P03 could not obtain lock on row in relation "bill" after 0.00 s
RESULT (lock_timeout = 500ms): ERROR 55P03 canceling statement due to lock timeout after 0.50 s
S5  ARCHIVING NOVEMBER 2025 - a 2 s energy report is running; meters keep sending readings every 50 ms
RESULT (ALTER TABLE ... DETACH PARTITION): 21 readings inserted; slowest insert 1963 ms, median 1.4 ms; partitions now attached = 12
RESULT (ALTER TABLE ... DETACH PARTITION CONCURRENTLY): 59 readings inserted; slowest insert 7 ms, median 1.6 ms; partitions now attached =
S6  MONTH-END REPORT - total revenue, then revenue by channel, while the app keeps selling tokens
RESULT (READ COMMITTED): total line 379,301,000.00; sum of channel lines 379,386,000.00 => REPORT DOES NOT ADD UP (difference 85,000.00)
RESULT (REPEATABLE READ): total line 379,392,000.00; sum of channel lines 379,392,000.00 => totals agree
```

Figure 3. Concurrency scenarios S1–S6: duplicate payment, bill payment, tariff change, lock waits, partition archiving, month-end report (Portfolio 3)

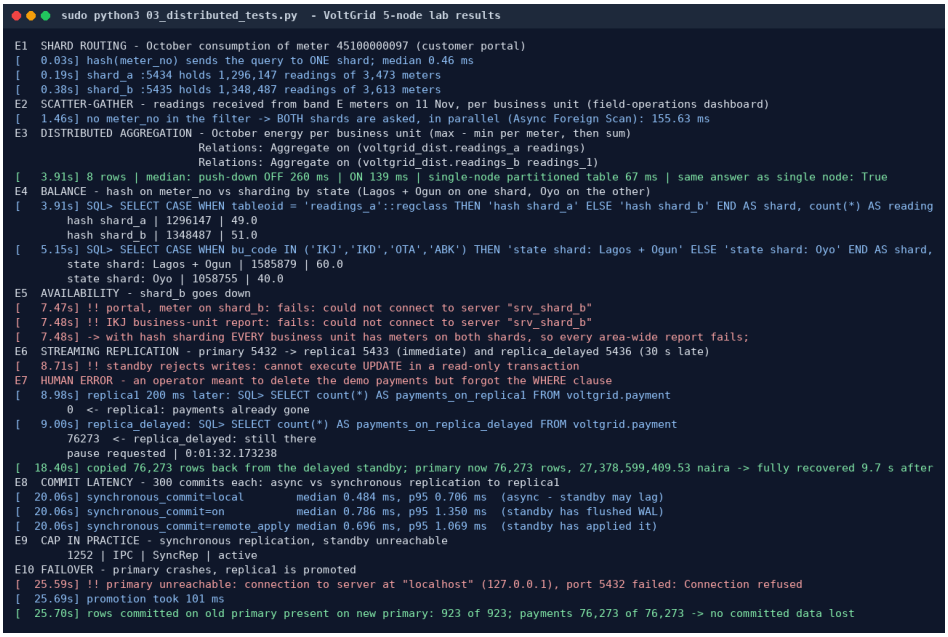


Figure 4. Distributed experiments E1–E10, including recovery from a DELETE without WHERE using the delayed standby (Portfolio 5)

All raw outputs, execution plans, scripts and per-portfolio READMEs are in the GitHub repository; each README states how the activity was implemented and tested and how to reproduce it.

4. Individual technical reflection

Design choices. I chose an electricity distributor because it mixes money with very large volumes of measurement data, and each needs different treatment. For money I let the database decide: a trigger prices every token from the tariff valid that day, a unique payment reference makes vending idempotent, and CHECK and exclusion constraints stop overpaid bills and overlapping tariff versions. For readings I chose monthly range partitioning, so that queries on one month and archiving of old months stay cheap. Work orders, whose fields differ by job type, went into documents, and fast operational signals went into Redis. For distribution I hash-sharded readings by meter so that each meter's history stays on one server, and added a delayed standby because the most likely data loss is a human mistake, not a disk failure.

Challenges. My first version of the revenue-protection query computed the last purchase of every meter before filtering, so adding an index changed nothing; I only understood why after reading the plan and rewriting the query as an anti-join. A partitioned table did not prune partitions until I stopped wrapping the date column in a function. Testing archiving showed something I did not expect: a DETACH PARTITION that is only waiting for a lock already blocks every new meter reading behind it. Getting the transaction scripts to clean up after themselves (tariff, bill and partitions restored) also took several attempts.

Lessons learned. Constraints are the cheapest protection: the unique payment reference turned a duplicate bank notification from a double token into a harmless repeat, and an exclusion constraint's index even sped up a report. The isolation level must match the job: one sale is fine at READ COMMITTED, but a quote followed by a charge, or a report of several queries, needs one snapshot. Distribution is not automatically faster; on one host the single partitioned table beat the shards, so sharding should wait until one server is not enough. Replication protects against hardware failure, while a delayed standby and backups protect against people.

5. References

- Connolly, T. and Begg, C. (2015) Database Systems. 6th edn. Pearson; Elmasri, R. and Navathe, S. B. (2016) Fundamentals of Database Systems. 7th edn. Pearson.
- Kleppmann, M. (2017) Designing Data-Intensive Applications. O'Reilly Media (chapters on transactions, replication, partitioning and distributed commit).
- PostgreSQL Global Development Group (2024) PostgreSQL 16 Documentation. postgresql.org/docs/16.
- Redis Ltd. Redis documentation (redis.io/docs); MongoDB Inc. MongoDB Manual (mongodb.com/docs/manual).
- Nigerian Electricity Regulatory Commission (2024) Multi-Year Tariff Order (service-based tariff bands); Federal Republic of Nigeria (2023) Nigeria Data Protection Act.

6. AI use declaration

Tool: Claude (Anthropic), used through the Claude app. **Used for:** drafting the SQL and Python scripts, generating the synthetic data set, running the experiments in a sandboxed environment, and drafting the README files and this documentation. **Review and verification:** I re-ran the scripts on my own PostgreSQL and Redis installation, checked that the outputs, plans and figures in the evidence folders match my runs, compared each README with the results, and rewrote the explanations in my own words. I can explain and reproduce every part of the portfolio.